Assignment Solution: Architecting and Implementing a Resilient Global Telemetry Platform

Prepared by: Somya Jaiswal **Roll Number:** 2025EM1100101

Part 1: System Architecture & Data Paradigms

1. Scaling Strategy Relying on a single-node system for a fleet of 500,000 vehicles streaming telemetry 24/7 hits the "Wall" in hardware limitations. Vertical scaling (Scaling-Up) involves adding more CPU, RAM, and disk space to a single server. Eventually, this hits a physical ceiling where motherboards can no longer support additional components, **PCIe bus lanes become fully saturated, and thermal thresholds prevent further continuous processing.** The financial cost becomes exponential, and a single node represents a single point of failure.

Horizontal scaling (Scaling-Out) is strictly required for this platform due to the Three Vs of Big Data: * **Volume:** 500,000 vehicles streaming continuously generates petabytes of data. Scale-out architecture allows us to distribute this storage across thousands of cheap, commodity hardware nodes (e.g., using HDFS or S3). * **Velocity:** Data arrives 24/7 at a massive rate. A single processor cannot ingest this network I/O. A distributed cluster allows parallel ingestion across multiple worker nodes. * **Variety:** Telemetry often combines structured metrics (speed, battery) with semi-structured logs (location streams). Distributed computing frameworks can handle varying formats without rigid upfront schemas.

2. Consistency Models When dealing with the high-velocity ingestion of vehicle coordinates, we must evaluate the CAP Theorem (Consistency, Availability, Partition Tolerance). In distributed networks, network partitions (P) are inevitable, leaving us to choose between Consistency (C) and Availability (A).

- **ACID (Atomicity, Consistency, Isolation, Durability):** Prioritizes strong consistency. If a node fails, the system locks database tables to ensure all nodes reflect the exact same data before accepting new writes. This is disastrous for high-velocity streaming, as locking causes massive ingestion bottlenecks.
- **BASE (Basically Available, Soft state, Eventual consistency):** Prioritizes availability. The system accepts writes continuously, even if different nodes temporarily hold different versions of the data. Over time, the nodes synchronize (Eventual Consistency).

For this telemetry workload, the BASE model is the correct choice. We require a highly Available and Partition-Tolerant (AP) system. Missing a vehicle coordinate for a fraction of a second is acceptable (eventual consistency), but dropping thousands of incoming network packets because a database table is locked (ACID) would cripple the real-time monitoring system.

Part 2: Batch Processing & MapReduce

1. Logical Flow (MapReduce) To aggregate the total miles driven per vehicle model using MapReduce, the flow is natively distributed across the cluster: * **Split:** The massive batch dataset is divided into logical partitions (e.g., 128MB chunks) distributed across data nodes. * **Map Phase:** Worker nodes execute the map function on their local splits. The mapper parses the record and outputs key-value pairs: `(vehicle_model, miles_driven)`. * **Shuffle Phase:** The network physically routes and groups all intermediate key-value pairs sharing the same key `(vehicle_model)` to the same reducer node. * **Sort Phase:** The intermediate data is sorted by key on the reducer node. * **Reduce Phase:** The reducer iterates through the values for each key, summing the miles driven to output the final aggregated result: `(vehicle_model, total_miles)`.

2. Hadoop vs. Spark for Iterative Algorithms Hadoop MapReduce relies heavily on disk-bound I/O. After every single Map and Reduce phase, Hadoop writes the intermediate results back to physical disk (HDFS) for fault tolerance. When running iterative machine learning algorithms (like K-Means or gradient descent) that pass over the same data hundreds of times, writing and reading from disk during every pass causes severe bottlenecks.

Spark overcomes this by utilizing an in-memory computing model. Spark caches intermediate working datasets in RAM (Random Access Memory) via Resilient Distributed Datasets (RDDs). By keeping the data in memory across iterations, Spark eliminates the massive latency of disk I/O, allowing iterative ML algorithms to process data orders of magnitude faster than Hadoop.

Part 3: PySpark Implementation & Resilience

*(**Note to Student:** Insert a screenshot of your terminal executing the Docker container or AWS environment here to fulfill the documentation requirement).*

PySpark Code Implementation:

```
from pyspark.sql import SparkSession
import pyspark.sql.functions as F

# Initialize Spark Session
spark = SparkSession.builder \
    .appName("ResilientTelemetryPlatform") \
    .config("spark.sql.shuffle.partitions", "200") \
    .getOrCreate()

# 3.4 Checkpointing Setup: Define HDFS or S3 directory for checkpointing
spark.sparkContext.setCheckpointDir("/tmp/spark_checkpoints")

# 3.1 Transformations and Actions (Ingestion)
# Narrow Dependency: Reading data involves no network shuffle.
df = spark.read.csv("telemetry_data.csv", header=True, inferSchema=True)
```

```
# 3.2 Optimization: Mitigating severe data skew with a Salting Strategy
# We append a random integer (0-9) to the vehicle model to distribute the
heavily skewed logs across multiple partitions.
# Narrow Dependency: withColumn acts on data locally on each node.
SALT_BINS = 10
salted_df = df.withColumn("salt", F.floor(F.rand() * SALT_BINS))

# Phase 1 of Aggregation (Wide Dependency: GroupBy triggers a Shuffle)
# Hash partitioning routes data based on the hash of (vehicle_model, salt).
partial_agg = salted_df.groupBy("vehicle_model", "salt").agg(
    F.sum("engine_heat").alias("sum_heat"),
    F.count("engine_heat").alias("count_heat")
)

# Phase 2 of Aggregation (Wide Dependency: Second Shuffle)
# We group by just the vehicle model to combine the salted bins for the final
average.
final_agg = partial_agg.groupBy("vehicle_model").agg(
    (F.sum("sum_heat") / F.sum("count_heat")).alias("avg_engine_heat")
)

# 3.4 Checkpointing Execution: Truncating the DAG
final_agg = final_agg.checkpoint()

# Action (Triggers Lazy Evaluation)
final_agg.show()
```

Architectural Explanations: * **Narrow vs. Wide Dependencies (3.1):** Transformations like `.read` and `.withColumn` are Narrow Dependencies; the parent partition maps 1:1 to a child partition without moving data across the network. `.groupBy()` is a Wide Dependency; a single parent partition must send data to multiple child partitions, forcing a network shuffle. * **Data Skew & Salting (3.2):** Because specific trucks generate 1000x more logs, standard grouping would route 99% of the data to a single worker node, crashing it (Out of Memory). By adding a "salt" (a random number 0-9), we artificially create 10 sub-keys for the heavy truck, distributing the processing across 10 nodes. **We utilize Hash Partitioning rather than Range Partitioning because our grouping keys (Vehicle Model + Salt) are distinct, categorical values rather than continuous numeric ranges. Hash Partitioning ensures these discrete keys are evenly mathematically distributed across the cluster.** We then do a final small aggregation to sum the sub-results. * **Fault Tolerance (3.3):** Spark achieves fault tolerance through Resilient Distributed Datasets (RDDs) and Lineage. Instead of copying data across the network, Spark logs the deterministic steps (the lineage) used to build a dataset. If a partition is lost due to a node failure, Spark simply looks at the lineage graph and re-executes the specific transformations on the source data to rebuild the missing partition locally. * **Checkpointing (3.4):** Implemented via `final_agg.checkpoint()`, this forces Spark to write the current DataFrame to reliable storage (like HDFS or S3). We discuss this deeply in Part 4.

Part 4: Advanced Execution Mechanics & Resilience Strategies

1. The Execution Model & DAGs Spark utilizes Lazy Evaluation. When writing transformations (like `.filter` or `.groupBy`), Spark does not process the data immediately. Instead, it logs the intent and builds a logical execution plan. Only when an Action (like `.show()` or `.write()`) is called does the Spark Catalyst Optimizer step in. Upon execution, Spark constructs a Directed Acyclic Graph (DAG). The DAG breaks the logical plan into physical Stages. Stage boundaries are explicitly drawn at Wide Dependencies (shuffles). Spark will aggressively pipeline as many Narrow Dependencies as possible into a single task within a Stage, only halting to execute a Stage when network I/O (a shuffle) is absolutely required.

2. Data Locality & Fault Tolerance Network congestion is a massive bottleneck in distributed systems. Spark adheres to the "Don't move data, move code" philosophy (Data Locality). Because data blocks are massive and code binaries are tiny, Spark schedules computational tasks (the code) on the exact worker nodes where the target data blocks already reside on disk or in RAM. Unlike Hadoop, which heavily taxes I/O bandwidth by replicating data 3x across the cluster for safety, Spark relies on Lineage-based recovery. Spark maintains stability by remembering the "recipe" of transformations. If a node dies, no data was duplicated; Spark simply routes the "recipe" to another node holding a replica of the source block and re-computes the lost partition.

3. Mitigating Lineage Liability In highly iterative scenarios (like predictive maintenance ML models running hundreds of times on telemetry data), Spark faces the "Liability of Lineage". Because the lineage graph (the family tree of transformations) grows with every iteration, the DAG becomes massive. This causes two critical failures: * **StackOverflow Errors:** The Spark driver physically runs out of memory trying to traverse and resolve the impossibly long DAG execution plan. * **Degraded Recovery:** If a node fails at iteration 500, Spark must rebuild that data by re-executing all 500 preceding iterations from the source, causing an unacceptable recovery delay.

Checkpointing solves this. While standard Spark caching (`.cache()`) stores data in memory for fast access, it retains the lineage graph (if the cache is lost, it must compute from scratch). Checkpointing, however, saves the dataset to reliable, persistent storage (HDFS/S3) and completely severs the lineage tree (truncates the DAG). By checkpointing every 50 iterations, we ensure the DAG never grows long enough to cause StackOverflows, and in a failure, Spark only has to rebuild data from the most recent checkpoint, vastly stabilizing recovery time.